DESAFIO BALANCEO DE CARGA HAProxy + Docker + React + FastAPI + JMeter



Integrantes

-

-

Daniela Ussa Villa - 2246449

Docente

HEBERTH FABIAN MARTINEZ VELASQUEZ

Grupo

REDES E INFRAESTRUCTURA 51 - (2026-2S)

Universidad Autónoma de Occidente

Facultad de Ingeniería y Ciencias Básicas

Santiago de Cali, 13 de octubre de 2026

Resumen

Se documenta una práctica de contenerización, comunicación entre servicios y balanceo de carga. La solución integra una interfaz web desarrollada con React, dos instancias de un servicio FastAPI, HAProxy como proxy inverso y balanceador, y Docker Compose como herramienta de orquestación local. Se realizaron pruebas con Apache JMeter para tres escenarios de carga: 10, 50 y 100 usuarios virtuales. Las capturas suministradas evidencian el despliegue de los contenedores, la interfaz web, la configuración de HAProxy y la ejecución de las pruebas. El análisis numérico se limita a los valores legibles y confirmados; los restantes deben validarse en los reportes originales de JMeter.

1. Introducción

Una aplicación web puede recibir solicitudes simultáneas de múltiples clientes. Cuando todas las peticiones se concentran en una sola instancia, la capacidad del servidor puede convertirse en un límite para el servicio. El balanceo de carga permite distribuir las solicitudes entre varias instancias disponibles, con el objetivo de aprovechar mejor los recursos y favorecer la disponibilidad.

En esta práctica se emplea HAProxy para dirigir el tráfico hacia dos instancias de FastAPI mediante la estrategia roundrobin. Además, se utiliza Apache JMeter para simular distintos niveles de carga y observar los tiempos de respuesta, la tasa de errores y el rendimiento del servicio.

2. Objetivos

Objetivo general

Implementar y evaluar una arquitectura web contenerizada con balanceo de carga mediante HAProxy, Docker, React y FastAPI, utilizando Apache JMeter para realizar pruebas de rendimiento.

Objetivos específicos

- Desplegar el frontend, dos instancias del backend y HAProxy mediante Docker Compose.
- Configurar el enrutamiento HTTP y la distribución de peticiones entre las instancias FastAPI.
- Comprobar el estado de los contenedores y la disponibilidad de la aplicación.
- Ejecutar pruebas con 10, 50 y 100 usuarios virtuales y documentar sus resultados.
- Analizar el comportamiento del sistema y conservar evidencias reproducibles.
-

3. Herramientas utilizadas

Herramienta	Función en la práctica
Git y GitHub	Clonación y control de versiones del repositorio.
Docker / Docker Compose	Construcción y ejecución de los servicios en contenedores.
React y Vite	Interfaz web del frontend.
FastAPI	Servicio HTTP que atiende solicitudes a la API.
HAProxy	Proxy inverso, distribución de carga y estadísticas.
Apache JMeter	Generación de carga y recopilación de resultados HTTP.
Java 21	Entorno requerido para ejecutar JMeter en Windows.

4. Arquitectura de la solución

El navegador o JMeter se comunica con HAProxy a través del puerto 8080 publicado en el equipo anfitrión. HAProxy distingue las rutas: las solicitudes cuyo prefijo es /api/ se envían al conjunto de backends FastAPI, mientras que el tráfico restante se dirige al frontend React. Los servidores de la API escuchan en el puerto 8000 dentro de la red de Docker; React utiliza el puerto 5173. La interfaz de estadísticas se encuentra en /stats.

Flujo de la API: cliente o JMeter → localhost:8080 → HAProxy → backend1 o backend2 → respuesta HTTP. Flujo del frontend: navegador → localhost:8080 → HAProxy → React.

5. Implementación y evidencias

5.1 Despliegue de contenedores

Se utilizó el comando **docker compose up --build** para construir y ejecutar los servicios definidos en docker-compose.yml. La captura muestra el entorno Docker Desktop con los componentes del proyecto. La comprobación mediante docker compose ps permite verificar el estado y los puertos publicados.

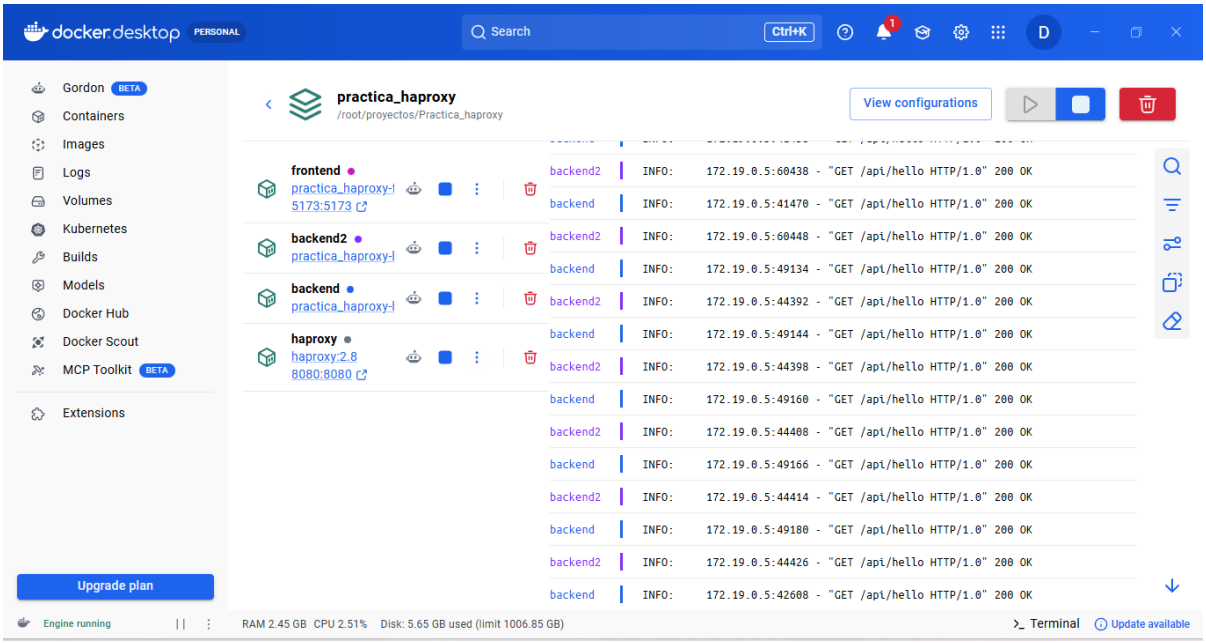


Figura 1. Contenedores del proyecto en Docker Desktop.

5.2 Interfaz web

La aplicación React presenta una interfaz sencilla con el título “React + FastAPI + HAProxy” y el mensaje recibido desde el servicio. La visualización de la página demuestra que el frontend está accesible; para validar el recorrido a través del balanceador se utiliza localhost:8080.

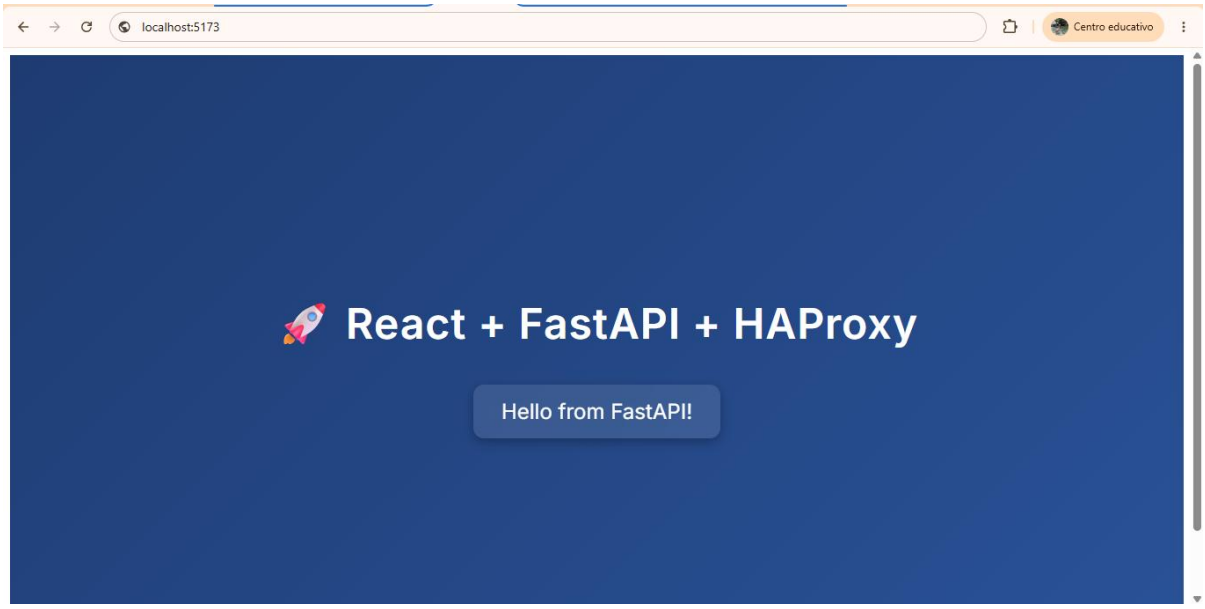


Figura 2. Interfaz React ejecutándose en el navegador.

5.3 Configuración del balanceador

El archivo haproxy.cfg define el listener HTTP en el puerto 8080, reglas de enrutamiento para /api/ y /stats, y dos destinos FastAPI. El algoritmo balance roundrobin alterna la

selección de servidores. La comprobación de salud utiliza GET /api/hello. La configuración de estadísticas permite consultar /stats.

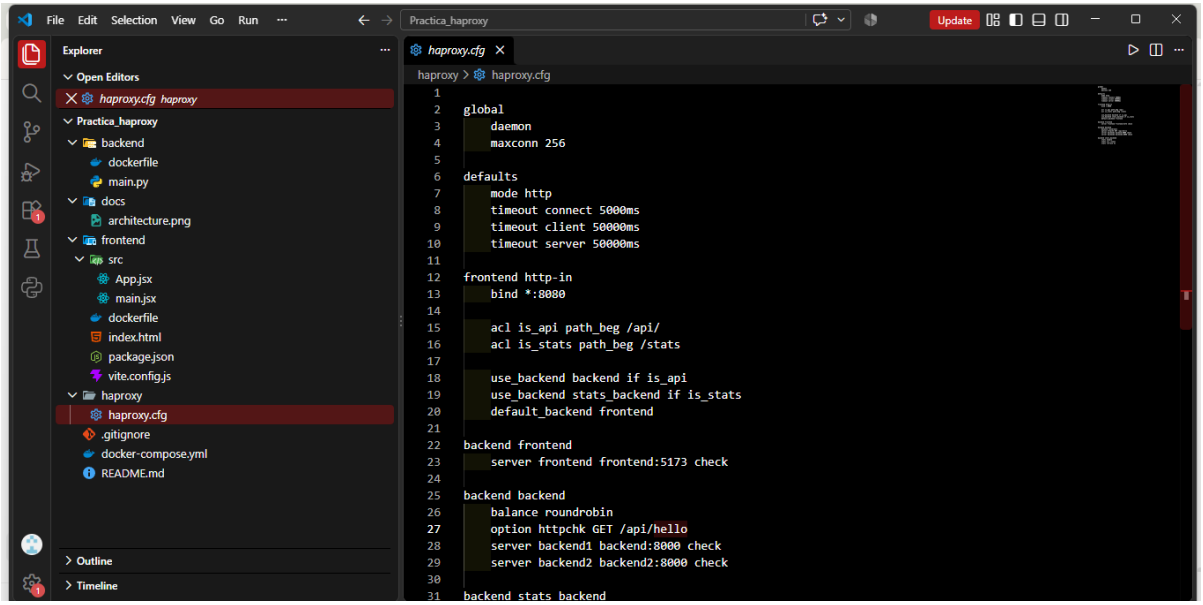


Figura 3. Archivo haproxy.cfg abierto en el editor.

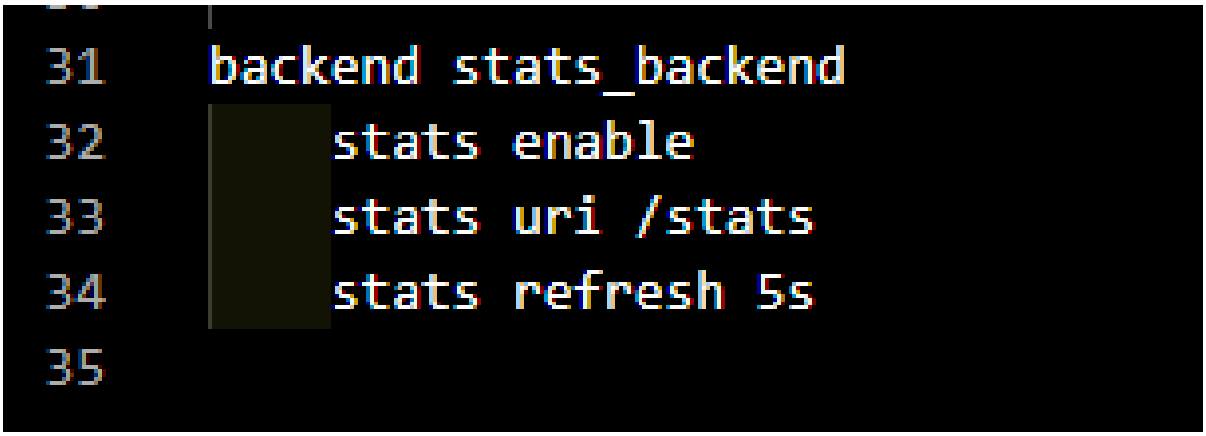
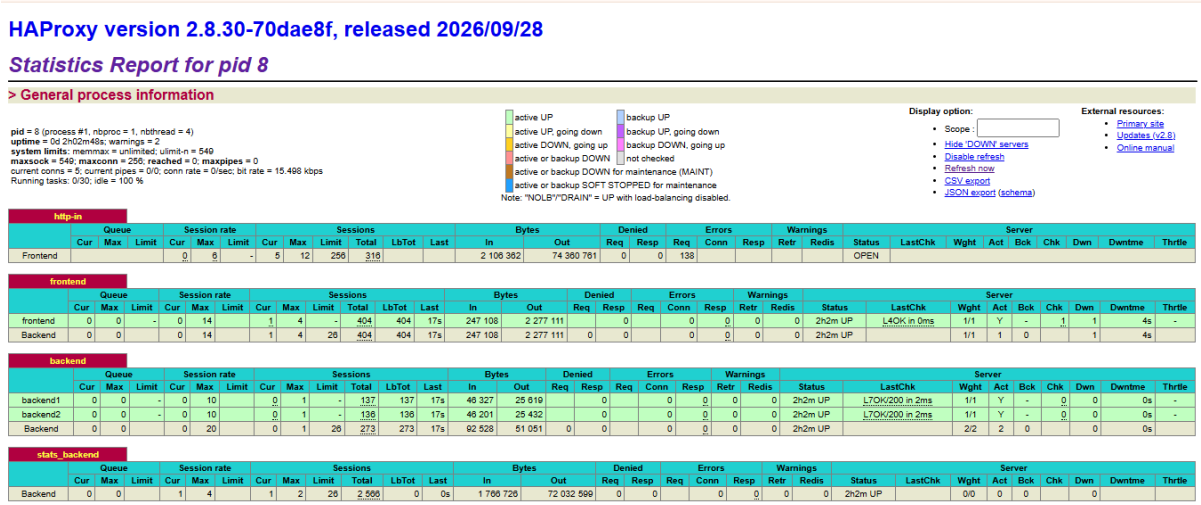


Figura 4. Detalle de configuración del panel de estadísticas de HAProxy.

5.4 Supervisión de HAProxy

El panel de estadísticas muestra el estado de los servidores y contadores acumulados de sesiones, errores y tráfico. Las capturas permiten documentar que los backends están registrados y operativos. Los contadores acumulados no representan exclusivamente una prueba JMeter si el panel recibió tráfico anterior.



HAProxy version 2.8.30-70dae8f, released 2026/09/28

Statistics Report for pid 8

> General process information

pid = 8 (process #1, nbproc = 1, nbthread = 4)
uptime = 0d 2h02m48s; warnings = 2
system limits: memmax = unlimited; ulimit-n = 549
maxsock = 549; **maxconn** = 256; **reached** = 0; **maxpipes** = 0
current conns = 5; current pipes = 0/0; conn rate = 0/sec; bit rate = 15.498 kbps
Running tasks: 0/30; idle = 100 %

active UP
active UP, going down
active DOWN, going up
active or backup DOWN
active or backup DOWN for maintenance (MAINT)
active or backup SOFT STOPPED for maintenance
backup UP
backup UP, going down
backup DOWN, going up
not checked
Note: "NOLB"/"DRAIN" = UP with load-balancing disabled.

Display option:
- Scope :
- Hide 'DOWN' servers
- Disable refresh
- Refresh now
- CSV export
- JSON export (schema)

External resources:
- Primary site
- Updates (v2.8)
- Online manual

http-in

	Queue			Session rate			Sessions						Bytes		Denied		Errors		Warnings		Server								
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
Frontend				0	6	-	5	12	256	316			2 106 362	74 360 761	0	0	138					OPEN								

frontend

	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Server							
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
frontend	0	0	-	0	14		1	4	-	404	404	17s	247 108	2 277 111		0		0	0	0	0	2h2m UP	L4OK in 0ms	1/1	Y	-	1	1	4s	-
Backend	0	0		0	14		1	4	26	404	404	17s	247 108	2 277 111	0	0		0	0	0	0	2h2m UP		1/1	1	0		1	4s	

backend

	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Server							
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
backend1	0	0	-	0	10		0	1	-	137	137	17s	46 327	25 619		0		0	0	0	0	2h2m UP	L7OK/200 in 2ms	1/1	Y	-	0	0	0s	-
backend2	0	0	-	0	10		0	1	-	136	136	17s	46 201	25 432		0		0	0	0	0	2h2m UP	L7OK/200 in 2ms	1/1	Y	-	0	0	0s	-
Backend	0	0		0	20		0	1	26	273	273	17s	92 528	51 051	0	0		0	0	0	0	2h2m UP		2/2	2	0		0	0s	

stats_backend

	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Server							
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
Backend	0	0		1	4		1	2	26	2 566	0	0s	1 766 726	72 032 599	0	0		0	0	0	0	2h2m UP		0/0	0	0		0		

Figura 5. Panel de estadísticas de HAProxy durante la práctica.

6. Metodología de las pruebas de carga

Se diseñó un plan de pruebas HTTP en Apache JMeter. El sampler realiza solicitudes GET a http://localhost:8080/api/hello para atravesar HAProxy antes de llegar a FastAPI. Se añadieron los listeners View Results Tree, Summary Report y Response Time Graph. En la configuración utilizada durante la práctica, cada usuario ejecuta diez iteraciones y el periodo de incorporación progresiva es de diez segundos.

Escenario	Usuarios virtuales	Ramp-up	Iteraciones por usuario	Solicitudes previstas
Prueba 1	10	10 s	10	100
Prueba 2	50	10 s	10	500
Prueba 3	100	10 s	10	1.000

Estas cifras corresponden a solicitudes previstas si cada iteración ejecuta una única petición y la prueba finaliza sin interrupciones. El número de usuarios virtuales no equivale automáticamente al mismo número de solicitudes exactamente simultáneas.

6.1 Evidencias del escenario de 10 usuarios

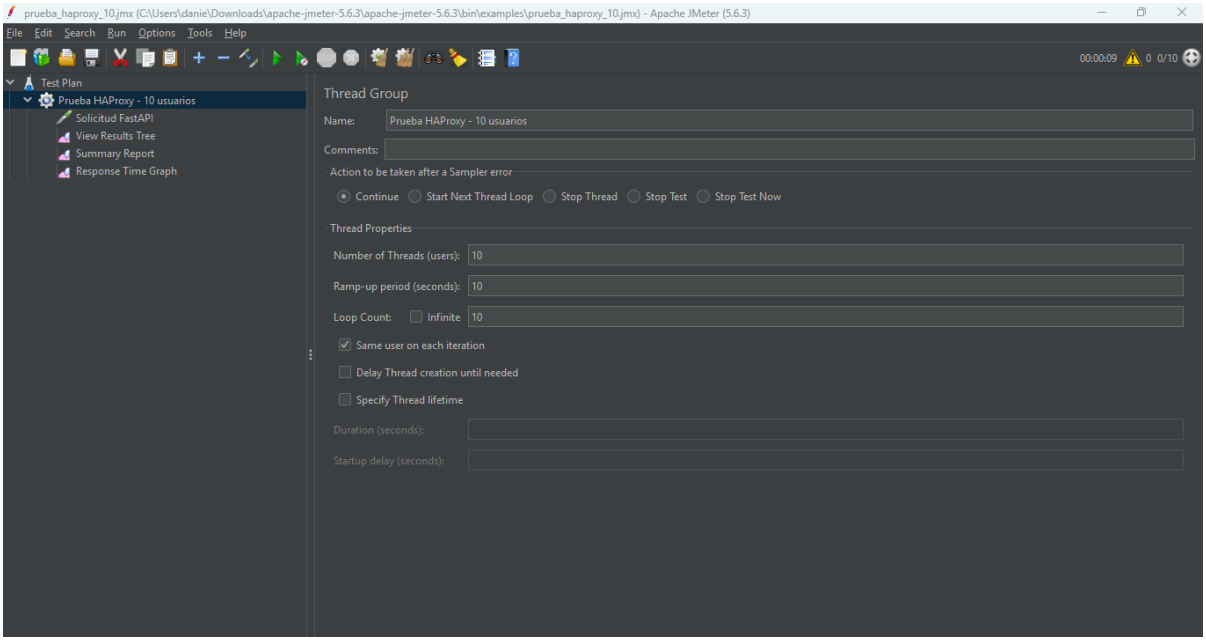


Figura 6. Configuración de JMeter para la primera prueba de carga.

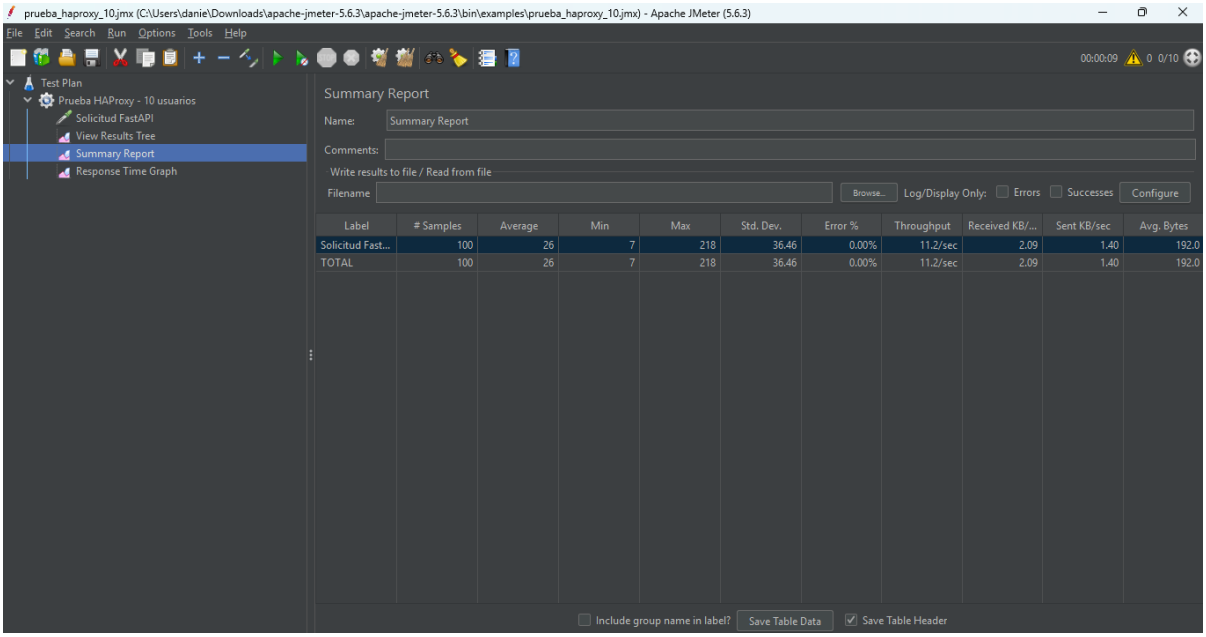


Figura 7. Summary Report de la primera prueba.

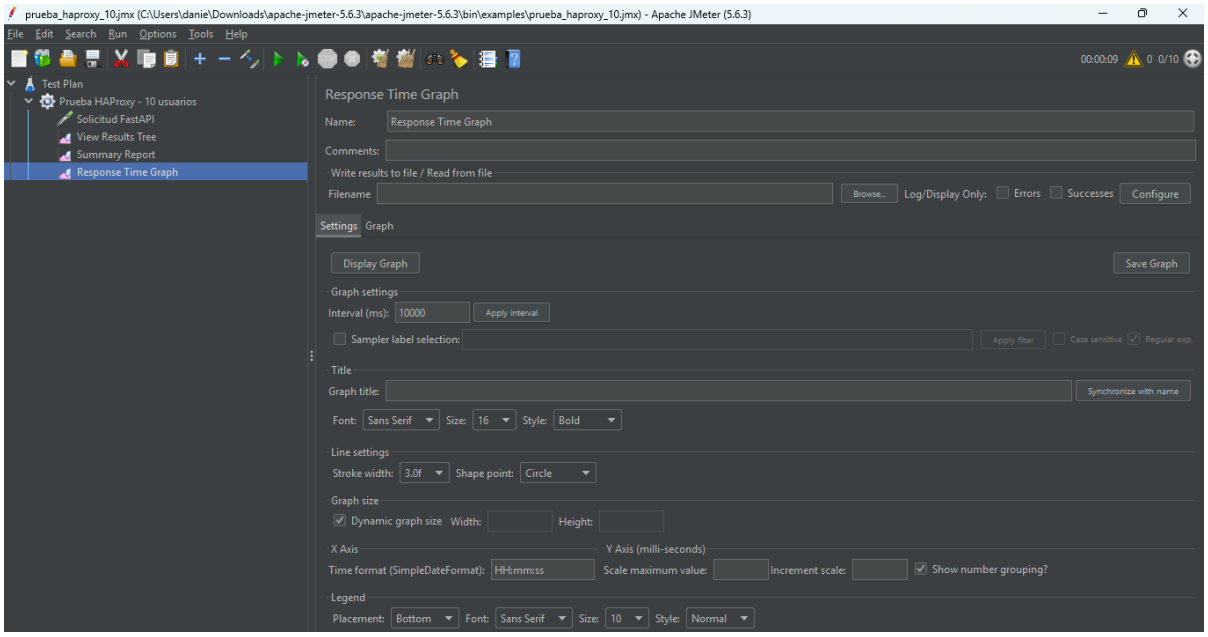


Figura 8. Configuración del listener Response Time Graph en JMeter.

6.2 Evidencias del escenario de 50 usuarios

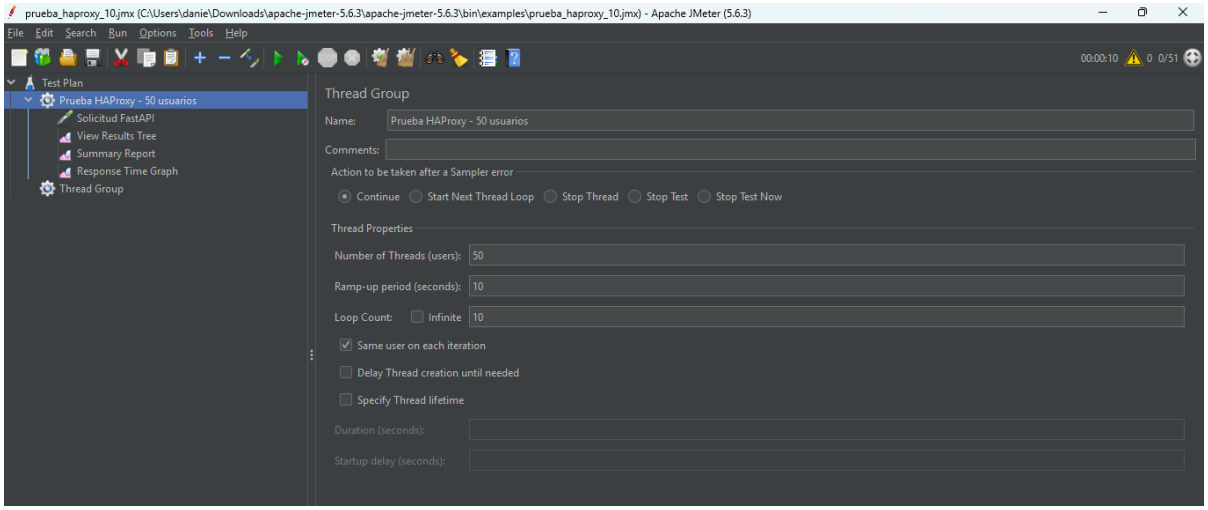


Figura 9. Configuración de JMeter correspondiente al escenario de 50 usuarios.

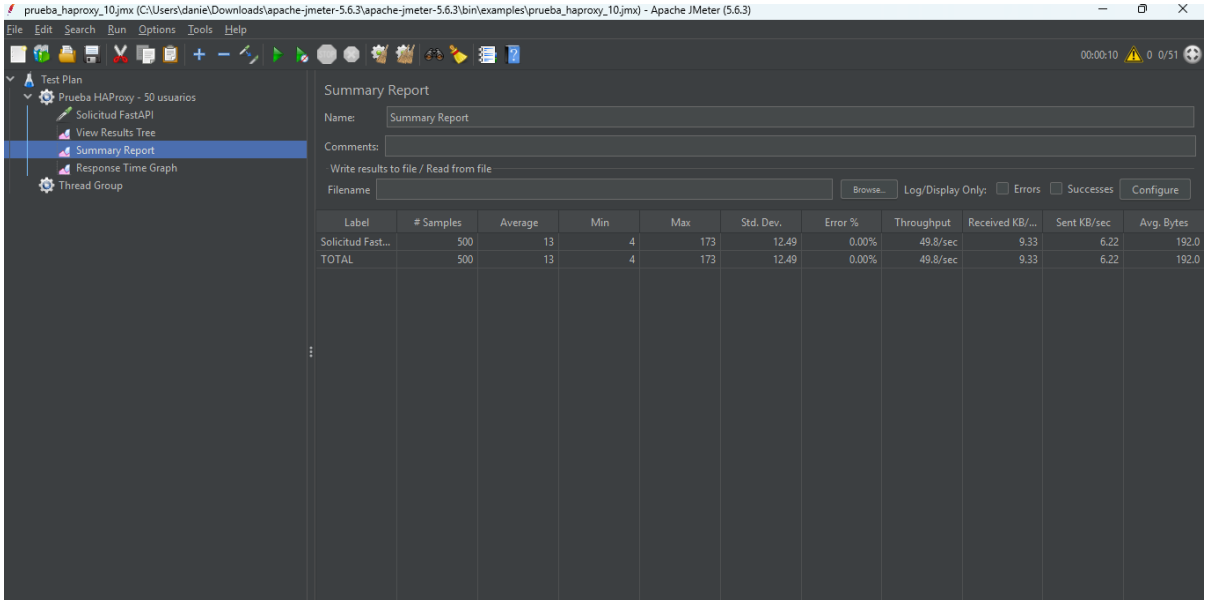


Figura 10. Summary Report capturado para el escenario de 50 usuarios.

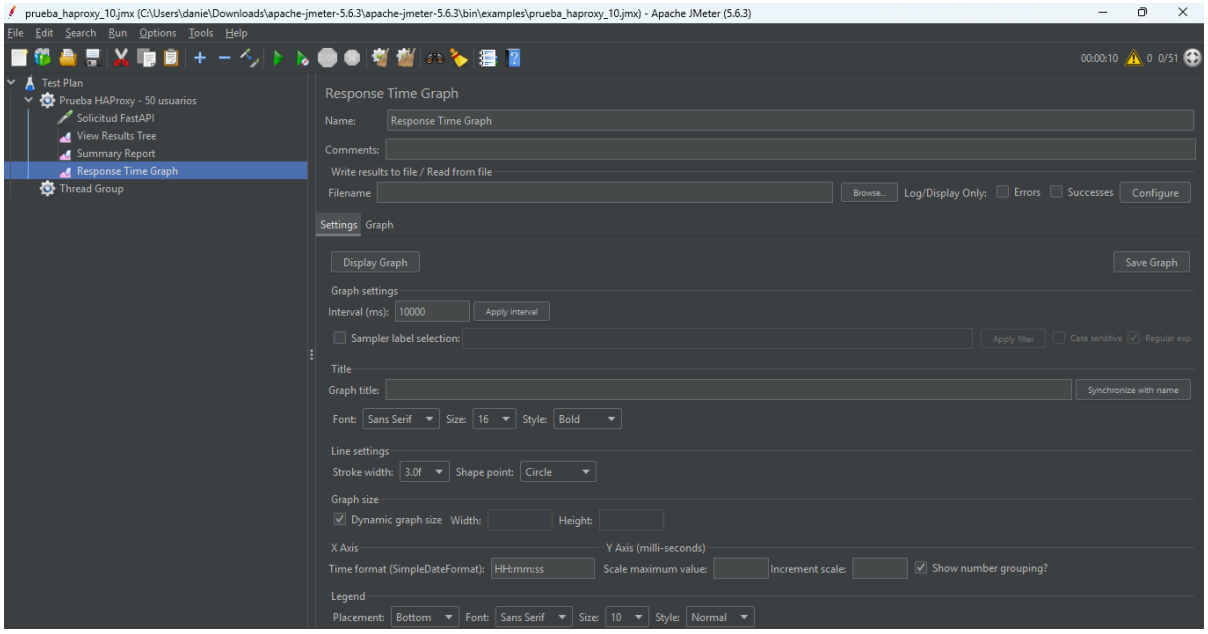


Figura 11. Pantalla de Response Time Graph correspondiente a las pruebas.

6.3 Evidencias del escenario de 100 usuarios

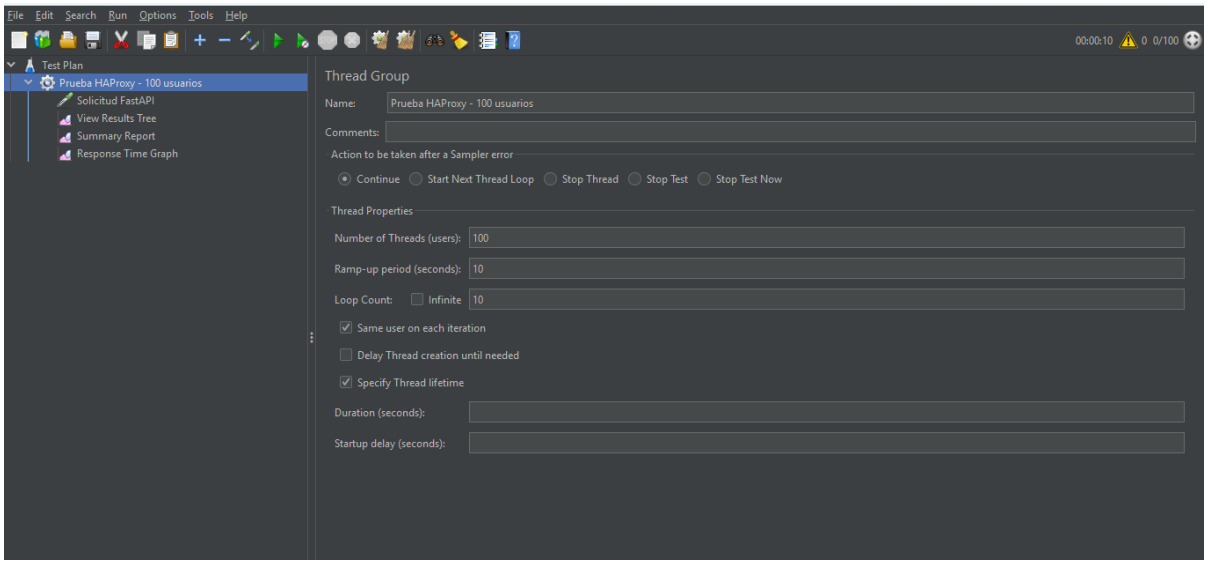


Figura 12. Configuración de JMeter correspondiente al escenario de 100 usuarios.

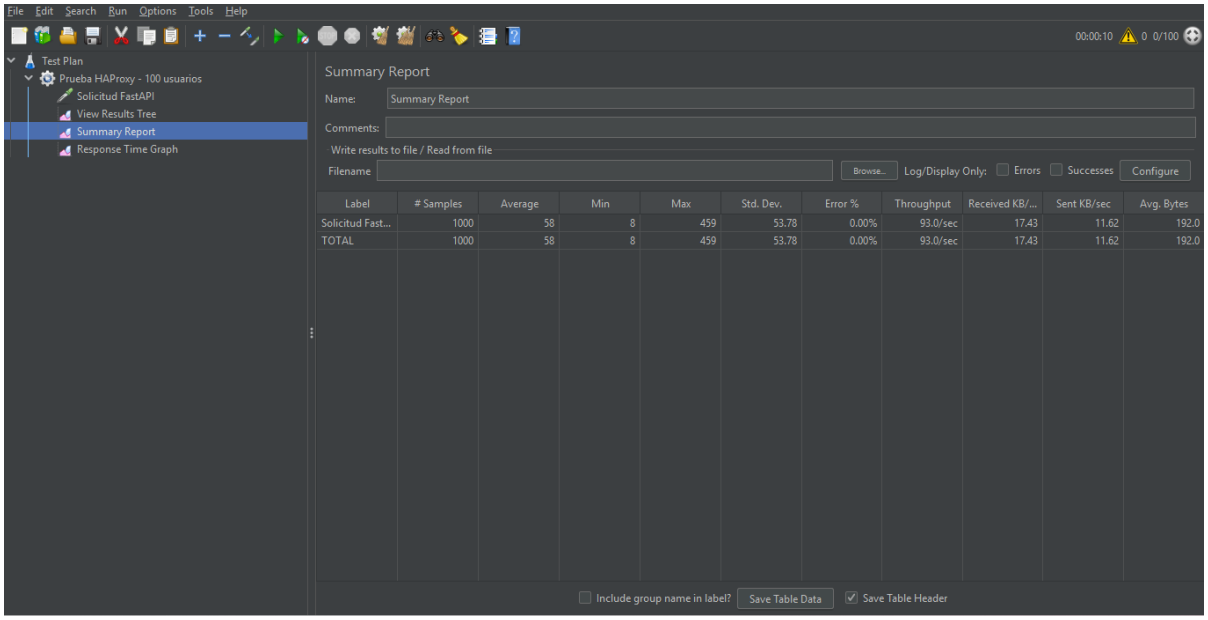
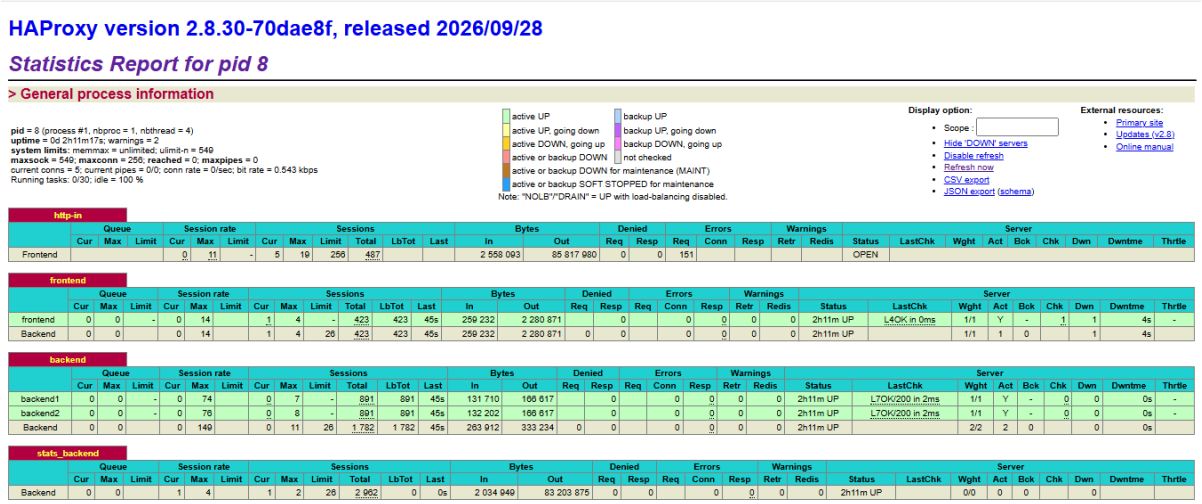


Figura 13. Summary Report del escenario de 100 usuarios.



HAProxy version 2.8.30-70dae8f, released 2026/09/28

Statistics Report for pid 8

> General process information

pid = 8 (process #1, nbproc = 1, nbthread = 4)
uptime = 0d 2h11m17s; warnings = 2
system limits: memmax = unlimited; ulimit-n = 549
maxsock = 549; **maxconn** = 256; **reached** = 0; **maxpipes** = 0
current conns = 5; current pipes = 0/0; conn rate = 0/sec; bit rate = 0.543 kbps
Running tasks: 0/30; idle = 100 %

active UP
active UP, going down
active DOWN, going up
active or backup DOWN
active or backup DOWN for maintenance (MAINT)
active or backup SOFT STOPPED for maintenance
backup UP
backup UP, going down
backup DOWN, going up
not checked
Note: "NOLB"/"DRAIN" = UP with load-balancing disabled.

Display option:
- Scope :
- Hide 'DOWN' servers
- Disable refresh
- Refresh now
- CSV export
- JSON export (schema)

External resources:
- Primary site
- Updates (v2.8)
- Online manual

http-in

	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Server									
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
Frontend				0	11	-	5	19	256	487			2 558 093	85 817 980	0	0	151					OPEN									

frontend

	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Server									
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
frontend	0	0		0	14		1	4	-	423	423	45s	259 232	2 280 871		0		0	0	0	0	2h11m UP	L4OK in 0ms	1/1	Y	-	1	1	4s	-
Backend	0	0		0	14		1	4	26	423	423	45s	259 232	2 280 871	0	0		0	0	0	0	2h11m UP		1/1	1	0		1	4s	

backend

	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Server									
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
backend1	0	0	-	0	74		0	7	-	891	891	45s	131 710	166 617		0		0	0	0	0	2h11m UP	L7OK/200 in 2ms	1/1	Y	-	0	0	0s	-
backend2	0	0	-	0	76		0	8	-	891	891	45s	132 202	166 617		0		0	0	0	0	2h11m UP	L7OK/200 in 2ms	1/1	Y	-	0	0	0s	-
Backend	0	0		0	149		0	11	26	1 782	1 782	45s	263 912	333 234	0	0		0	0	0	0	2h11m UP		2/2	2	0		0	0s	

stats_backend

	Queue			Session rate			Sessions						Bytes		Denied		Errors			Warnings		Server									
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntme	Thrtle
Backend	0	0		1	4		1	2	26	2 962	0	0s	2 034 949	83 203 875	0	0		0	0	0	0	2h11m UP		0/0	0	0		0		

Figura 14. Estadísticas de HAProxy capturadas después de las pruebas.

7. Resultados y análisis

El Summary Report de la prueba de 10 usuarios, compartido también durante el desarrollo de la práctica, registró 100 solicitudes, un promedio de 26 ms, mínimo de 7 ms, máximo de 218 ms, desviación estándar de 36,46 ms, 0,00 % de errores y un rendimiento de 11,2 solicitudes por segundo. Las capturas insertadas en el documento original para 50 y 100 usuarios tienen una resolución insuficiente para transcribir sus cifras con seguridad; se conservan como evidencia visual, pero se requiere el dato original o una captura legible para completar la comparación cuantitativa.

Indicador	10 usuarios	50 usuarios	100 usuarios
Solicitudes	100	Por confirmar	Por confirmar
Promedio (ms)	26	Por confirmar	Por confirmar
Mínimo (ms)	7	Por confirmar	Por confirmar
Máximo (ms)	218	Por confirmar	Por confirmar
Desviación estándar (ms)	36,46	Por confirmar	Por confirmar
Errores (%)	0,00	Por confirmar	Por confirmar
Throughput (solicitudes/s)	11,2	Por confirmar	Por confirmar

7.1 Interpretación

En el primer escenario se atendieron 100 solicitudes sin errores registrados y con un tiempo medio de 26 ms. Esto confirma el funcionamiento del recorrido de la API bajo la carga evaluada, pero no permite concluir por sí solo cuál es el límite de capacidad del sistema. Para identificar tendencias al aumentar la carga es necesario comparar los promedios, máximos, porcentajes de error y throughput de los tres escenarios.

Las estadísticas de HAProxy mostradas en la práctica registraron dos backends activos y una distribución de sesiones cercana al equilibrio. Esto es consistente con la estrategia roundrobin, aunque los contadores incluyen actividad acumulada y no deben interpretarse como una medición aislada de JMeter.

7.2 Consideraciones de validez

- Las pruebas se realizaron en un entorno local; los resultados dependen de los recursos y la carga del equipo anfitrión.
- Los listeners gráficos pueden añadir sobrecarga. Para mediciones definitivas conviene ejecutar JMeter en modo no gráfico y guardar archivos JTL.
- Para comparar escenarios, se debe mantener constante el endpoint, las iteraciones, la configuración del entorno y registrar las condiciones de cada ejecución.
- Una captura de la configuración del gráfico no equivale a una gráfica de tiempos generada; conviene exportar la gráfica real desde JMeter.

8. Conclusiones

- Se logró desplegar una arquitectura web distribuida en contenedores con un frontend React, dos servicios FastAPI y un balanceador HAProxy.
- La configuración de HAProxy permitió encaminar solicitudes hacia los backends y consultar estadísticas de su estado.
- La prueba de 10 usuarios completó 100 solicitudes sin errores registrados y con un tiempo promedio de respuesta de 26 ms.
- Se ejecutaron escenarios adicionales de 50 y 100 usuarios; su comparación cuantitativa debe completarse con los valores legibles de sus respectivos Summary Report.
- La práctica permitió relacionar conceptos de contenerización, proxy inverso, balanceo de carga, supervisión y evaluación del rendimiento mediante usuarios virtuales.

Referencias técnicas

HAProxy Technologies. HAProxy Configuration Manual. https://docs.haproxy.org/

Apache Software Foundation. Apache JMeter User Manual. https://jmeter.apache.org/usermanual/

Docker Inc. Docker Compose documentation. https://docs.docker.com/compose/

FastAPI. Official documentation. https://fastapi.tiangolo.com/

React. Official documentation. https://react.dev/